ADVANCED PYTHON QUIZ — SOLUTIONS

Introduction to Data Science | Answer key and worked coding solutions

SECTION A: MCQ ANSWERS WITH EXPLANATIONS

Q1 — Answer: A) [[1, 9], [2]]

a.copy() creates a shallow copy: the outer list is new, but its nested lists are shared. Appending 9 through b[0] also changes a[0]. Reassigning b[1] only changes the outer list entry in b.

Q2 — Answer: A) [5, 3]

The slice starts at index 5 and moves backward by 2, stopping before index 1. It visits indices 5 and 3, whose values are 5 and 3.

Q3 — Answer: A) [(1, 0), (2, 1)]

For x=1, y=0 satisfies the odd-sum condition. For x=2, y=1 satisfies it; y=0 does not. The comprehension therefore returns these two tuples.

Q4 — Answer: A) [1] / [2] / [1, 3]

The default list cache is created once and reused between calls that omit cache. The second call supplies a separate list, so it does not affect the default list.

Q5 — Answer: A) None 5 None

get('a', 5) returns the existing value None; the default is used only when the key is missing. get('b', 5) returns 5, while d['a'] returns None.

Q6 — Answer: A) E, followed by F on the next line

int('4.2') raises ValueError because the string is not an integer-form string. The except block prints E, and finally prints F. The else block does not run.

Q7 — Answer: A) {'a': 3, 'b': 2}

Dictionary construction processes pairs in order. The later pair for key 'a' replaces its earlier value, so 'a' maps to 3.

Q8 — Answer: A) [(2, 'y'), (4, 'w')]

enumerate starts counting at 1. The comprehension keeps only even indices, which are 2 for 'y' and 4 for 'w'.

Q9 — Answer: A) 0 / [1, 4, 9] / []

next(g) consumes the first generated value, 0. The next list consumes the remaining values 1, 4, and 9. The generator is exhausted afterward, so the final list is empty.

Q10 — Answer: A) ['fig', 'pear', 'plum', 'kiwi']

Sorting by length puts 'fig' first. The other three words all have length 4, and Python's sort is stable, so their original relative order—pear, plum, kiwi—is preserved.

SECTION B: PYTHON CODING SOLUTIONS

Shared dataset used by all coding questions:

```
students = [
    {"name": "Ali", "allowance": 10000, "expenses": [1500, 2200, 1800, 3000], "category": "Regular"},
    {"name": "Sara", "allowance": 8000, "expenses": [2500, 3100, 2800], "category": "Scholarship"},
    {"name": "Usman", "allowance": 12000, "expenses": [1000, 1200], "category": "Regular"},
    {"name": "Ayesha", "allowance": 6000, "expenses": [2000, 2500, 1800], "category": "Scholarship"},
    {"name": "Zain", "allowance": 15000, "expenses": [4000, 5000, 4500], "category": "Regular"},
]
```

Q1 — Build a student summary

```
summary = []

for student in students:
    total_spent = sum(student["expenses"])
    balance = student["allowance"] - total_spent
    high_expenditure = total_spent > 0.80 * student["allowance"]

    summary.append({
        "name": student["name"],
        "total_spent": total_spent,
        "balance": balance,
        "high_expenditure": high_expenditure,
    })

print(summary)
```

Expected summary:

[

{'name': 'Ali', 'total_spent': 8500, 'balance': 1500, 'high_expenditure': True},

{'name': 'Sara', 'total_spent': 8400, 'balance': -400, 'high_expenditure': True},

{'name': 'Usman', 'total_spent': 2200, 'balance': 9800, 'high_expenditure': False},

{'name': 'Ayesha', 'total_spent': 6300, 'balance': -300, 'high_expenditure': True},

{'name': 'Zain', 'total_spent': 13500, 'balance': 1500, 'high_expenditure': True}

]

Explanation: sum() totals each expense list. A new dictionary is appended to summary, so the original student dictionaries are not changed.

Q2 — Calculate scholarship bonuses

```
scholarship_bonuses = {}

for student in students:
    if student["category"] == "Scholarship":
        balance = student["allowance"] - sum(student["expenses"])
        bonus = max(0, balance * 0.10)
        scholarship_bonuses[student["name"]] = round(bonus, 2)

print(scholarship_bonuses)
```

Expected output:

{'Sara': 0, 'Ayesha': 0}

Explanation: Sara's balance is 8000 - 8400 = -400, and Ayesha's is 6000 - 6300 = -300. Since each balance is negative, the bonus is capped at zero.

Q3 — Rank the highest-spending students

```
def rank_students(students, top_n=3):
    totals = [
        (student["name"], sum(student["expenses"]))
        for student in students
```

```
    ]
    totals.sort(key=lambda item: (-item[1], item[0]))
    return totals[:max(0, top_n)]

print(rank_students(students))
```

Expected output:

[('Zain', 13500), ('Ali', 8500), ('Sara', 8400)]

Explanation: The sort key uses negative spending to order totals from highest to lowest, then the name alphabetically for ties. A new list is sorted; the input list and its dictionaries are not reordered or modified.

Q4 — Record a purchase safely

```
def record_purchase(students, name, amount):
    # bool is a subclass of int, so reject it explicitly.
    if isinstance(amount, bool) or not isinstance(amount, (int, float)):
        return False
    if amount < 0:
        return False

    for student in students:
        if student["name"] == name:
            student["expenses"].append(amount)
            return True

    return False
```

Example:

print(record_purchase(students, "Sara", 500)) # True

print(record_purchase(students, "Unknown", 500)) # False

print(record_purchase(students, "Sara", True)) # False

print(record_purchase(students, "Sara", -10)) # False

Explanation: All validation happens before modifying the dataset. Only after a valid amount and exact matching name are found is the expense appended. Failed calls leave the dataset unchanged.

Q5 — Create a category-level report

```
def category_report(students):
    report = {}

    for student in students:
        category = student["category"]
        total = sum(student["expenses"])
        allowance = student["allowance"]

        if category not in report:
            report[category] = {
                "student_count": 0,
                "combined_expenses": 0,
                "over_80_percent_count": 0,
            }

        report[category]["student_count"] += 1
        report[category]["combined_expenses"] += total

        if total > 0.80 * allowance:
            report[category]["over_80_percent_count"] += 1

    highest_spending_category = min(
        report,
        key=lambda category: (
            -report[category]["combined_expenses"],
            category
        )
    )

    return report, highest_spending_category
```

```
print(category_report(students))
```

Expected report:

{

'Regular': {

'student_count': 3,

'combined_expenses': 24200,

'over_80_percent_count': 2

},

'Scholarship': {

'student_count': 2,

'combined_expenses': 14700,

'over_80_percent_count': 2

}

}

Highest-spending category: 'Regular'

Explanation: Each student updates the counters for their category. The threshold is strictly greater than 80%, not greater than or equal to 80%. The selection key prioritizes the largest combined total and then alphabetical order if totals tie.